



JTACS

DIGITAL COMMERCE & ADVISORY

BUILD & DELIVERY PLAN

Ref JTACS/PLN/2026-27/021

Date 10 September 2026

Revision 1.0

Status For approval

PREPARED BY

JTACS

Jnanottam

PREPARED FOR

Dr. R. Dhanush

Malnad produce — coffee, spices and estate goods

SUBJECT

Own Storefront & Owner Dashboard

A custom-built online store and an owner's dashboard, with WhatsApp community and ordering. Built on your own infrastructure — no platform rent and no commission on your sales.

01

What we are building

Three things, on infrastructure you own outright. Read this page if you read nothing else.

✓ **A storefront.**

Your own website where customers browse your coffee and spices, choose a pack size, and pay online. Fast on a phone, built to be found on Google, and built around where the produce comes from — provenance is what sells Malnad goods.

✓ **An owner dashboard.**

One screen that answers "how did we do today" — sales, orders waiting to be packed, what is selling, what is running out of stock, and a clean export for your accountant.

✓ **A WhatsApp channel.**

A community customers join for offers, plus the ability to order inside WhatsApp itself. Those orders arrive in the same dashboard automatically — nobody re-types them.

Why building your own is the right call here. A rented platform would charge roughly ₹21,000 a year in subscription plus a **2% cut of every order** for using an Indian payment gateway. On ₹1,50,000 of monthly sales that 2% alone is **₹42,480 a year**, rising every year you grow. Your own build has neither. You pay for hosting — a few thousand rupees a month — and nothing per sale. The trade is that the system is yours to look after, which is what Sections 06 to 08 are about.

One assumption to confirm before we build. This plan is engineered for a launch of **20–40 orders a day**, designed with headroom to **1,500 orders a day** so a festival rush or a post that travels does not take the store down. If you expect materially more than that from day one, tell us now — it changes the database and hosting tier, and it is far cheaper to decide before we build than after.

The storefront — what customers get

A · CATALOGUE BUILT FOR PRODUCE

- ❑ **Products with real variants.** One coffee, sold as 250 g / 500 g / 1 kg, and as beans / coarse / fine grind — each with its own price, stock count and barcode. Same for spices by weight. Customers pick on one page instead of hunting through six listings.
- ❑ **Collections** — Coffee, Spices, Honey & Estate Goods, Gift Boxes, and seasonal sets for harvest and festivals.
- ❑ **Provenance content.** Estate story, altitude, harvest month, roast date, single-origin notes. For Malnad produce this is not decoration — it is the reason someone pays your price instead of a supermarket's.
- ❑ **Search and filters** by category, weight, roast and price, with related-product suggestions on every page.
- ❑ **Gift and bulk options** — gift notes at checkout and a simple bulk-enquiry form for shops and offices.

B · CHECKOUT

- ❑ **Prepaid, through Razorpay** — UPI, cards, net banking and wallets. UPI carries no charge at all; cards cost 2% plus GST. Nothing ships before the money has cleared.

- ❑ **Guest checkout by default**, with an optional account for order history and faster repeat purchases. Forcing account creation is the single biggest cause of abandoned carts.
- ❑ **Delivery charges by pin code**, free above a cart value you choose, with your own delivery arrangement behind it.
- ❑ **GST-compliant invoice** generated automatically, with HSN codes and a sequential invoice number per financial year.

C · BUILT TO BE FOUND AND TO LOAD FAST

- ❑ **Mobile first.** Most of your traffic will be a phone on a patchy connection. Pages are built to open in about a second on 4G.
- ❑ **SEO from the start** — clean URLs, product structured data, sitemap, and a Google Business Profile, so "malnad coffee online" can find you.
- ❑ **Analytics** so you can see where visitors came from and what they bought.

03

The dashboard — what you get

Deliberately two layers. One is a full operations console; the other is the single screen you actually open with your morning coffee.

Layer 1

Operations console

The working tool for whoever runs the shop day to day.

- Orders — search, filter, open, mark packed and dispatched
- Products — add, edit, price, publish or hide
- Stock — counts per variant, low-stock warnings
- Refunds and cancellations, processed properly through Razorpay
- Customers and their order history
- Discount codes and offers

Layer 2

Owner View BESPOKE

One screen, built for you, answering the questions you actually ask.

- Sales today, this week, this month, against last
- Orders waiting to be packed — the number that matters at 9am
- Top products, and what is about to run out
- Website orders versus WhatsApp orders
- New versus repeat customers
- GST and HSN export for your accountant, per month

Both read the same database, so they can never disagree. The operations console is a proven open-source admin we configure and secure rather than write from scratch — that is deliberate, and Section 05 explains why. The Owner View is built specifically for you.

WhatsApp — community and ordering

You asked whether WhatsApp orders have to be typed into the dashboard by hand. They do not. Here is how they arrive on their own.

The rule that shapes this, and it is not negotiable. WhatsApp has two separate products. The free **WhatsApp Business App** is the only one that can run a Community. The paid **WhatsApp Business Platform (API)** is the only one that can broadcast without limit and send automatic order updates — and the moment a number moves to the API it can no longer use the Business App, and loses Communities entirely.

One number cannot do both. So we set up **two numbers** from day one, both branded as your business. Doing this later means losing a number's history, which is why it is decided now.

THREE WAYS AN ORDER CAN ARRIVE FROM WHATSAPP — ALL LAND IN THE DASHBOARD AUTOMATICALLY

ROUTE	HOW IT WORKS FOR THE CUSTOMER	HOW THE ORDER REACHES YOU	PHASE
Link out DAY ONE	They chat, and get a link to a ready-filled cart on the website. They pay there.	It is an ordinary website order. No extra machinery at all.	1
Order inside the chat	A form opens inside WhatsApp — pick products, quantity, address, confirm. No browser, no app switch.	WhatsApp posts the completed form to our server, which creates the order and marks its source as WhatsApp.	3
Pay inside the chat	Razorpay is linked to your WhatsApp business account, so UPI or card completes without leaving the conversation.	Razorpay confirms the payment to our server and the order is marked paid.	3
Manual entry	For phone orders and odd cases, staff create an order from the dashboard.	Always available — but the exception, not the workflow.	1

The in-chat form is a WhatsApp feature called Flows. There is one engineering constraint worth naming because it dictates how we build it: WhatsApp requires our server to acknowledge a submitted form **within three seconds** or the customer sees an error. So we accept the form instantly and do the real work — stock check, order creation, confirmation message — a moment later in the background. Section 06 covers that queue.

1. **Number 1 — Community**, on the free app: business profile, catalogue, an announcement group and topic groups for Harvest Updates, Offers, and Recipes & Brewing.
2. **Number 2 — ordering and updates**, on the API: Meta business verification, green tick application, and the shared inbox your staff work from.
3. **Message templates** submitted to Meta for approval: order confirmed, payment received, dispatched with tracking, delivered, abandoned-cart reminder, re-order reminder, and festival offers.
4. **Join links and QR codes** for the website, order confirmations, and a printed insert in every parcel — that insert is how a one-time buyer becomes a repeat one.
5. **Consent capture at every entry point**, recorded and timestamped. Meta bans numbers that message people who did not opt in, and a banned number is very hard to recover.

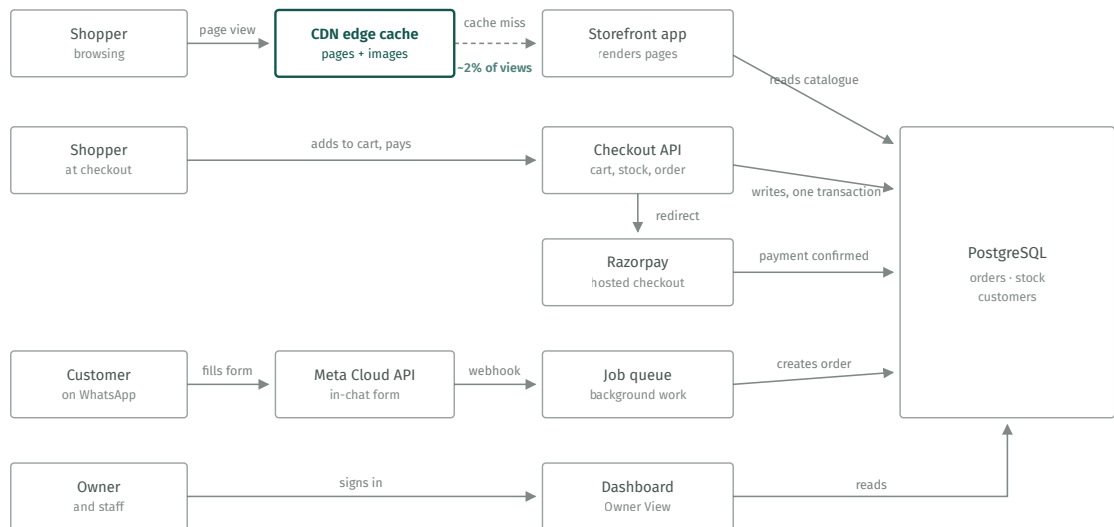
How it is built

The single most important engineering decision in this project is what we *do not* write ourselves.

We do not hand-write the shop engine. Carts, order states, stock locking, partial refunds — that is where custom stores fail, and none of it is unique to you. We build on **Medusa**, an open-source commerce core used in production worldwide, free and MIT-licensed, with no fees of any kind. It brings the order and stock logic already proven by thousands of stores, plus the operations console in Section 03. We then build what *is* yours: the storefront, the Owner View, and the WhatsApp layer. Fewer of our own bugs, and security fixes to the core come from its community rather than from us noticing.

THE PARTS

COMPONENTS AND WHAT EACH ONE IS RESPONSIBLE FOR		
PART	WHAT IT IS	WHY THIS ONE
Storefront	A Next.js website	Pages can be pre-built and cached, which is what makes the store fast and what absorbs traffic spikes
Commerce core	Medusa, self-hosted	Proven cart, order, stock and refund logic. No licence fee, no commission
Database	Managed PostgreSQL	The record of every order, product and customer. Managed, so backups and patching are handled
Cache & queue	Redis	Holds sessions, and holds background jobs so slow outside services never make a customer wait
Images	Object storage behind a CDN	Product photography served from servers near the customer, resized automatically
Payments	Razorpay hosted checkout	Card details are entered on Razorpay's page, never ours — so card data never touches your system
WhatsApp	Meta Cloud API	Direct from Meta. Order forms and automated updates
Watching it	Error tracking, uptime monitor, logs	We find out something broke before your customers tell you



Why a traffic spike does not take the shop down. Product and category pages are pre-built and served from cache at the network edge, so roughly 98% of browsing never reaches our application or the database at all — the top row terminates before it gets there. Only the paths that must change something (checkout, a WhatsApp order, a dashboard read) travel the full width. Those are a small fraction of traffic, and they are the only ones the database has to size for.

Capacity — will it hold up

You asked whether it can handle the load. Here are the numbers we design to, and the specific reasons each one holds.

DESIGN TARGETS, VERIFIED BY LOAD TEST BEFORE LAUNCH			
WHAT	EXPECTED AT LAUNCH	DESIGNED TO HANDLE	HEADROOM
Orders per day	20–40	1,500	~40×
Visitors per day	500–1,500	50,000	~35×
Shoppers on the site at once	20–60	2,000	~35×
Checkouts per minute	1–2	100	~50×
Speed and reliability			
Product page opens in	—	under 1.5 s	95% of visits
Checkout completes in	—	under 3 s	95% of orders
Uptime	—	99.9%	~43 min/month
Overselling incidents	—	Zero	prevented by design

1. **Browsing is served from cache, not from the database.** Product pages are pre-built and pushed to servers near the customer. A post that goes viral hits the cache, and the cache does not run out of breath. This is the single biggest reason the numbers above are achievable on modest hardware.
2. **Connection pooling on the database.** The classic way a small store dies under load is running out of database connections while the machine sits idle. A pooler sits in front so hundreds of simultaneous requests share a small, safe number of connections.
3. **Slow outside services never block a customer.** WhatsApp messages, confirmation emails and invoice generation go onto a background queue. If Meta is slow this evening, your customer still checks out in three seconds; the message goes a minute later.
4. **The application scales sideways automatically.** Traffic climbs, more copies of the app start; traffic falls, they stop. You pay for what you use, and a festival morning does not need anyone awake to handle it.
5. **Stock is decremented inside a locked transaction.** If two people buy the last 500 g pack in the same second, exactly one succeeds and the other is told it just sold out. This is a correctness guarantee, not a best effort.
6. **Rate limiting and bot protection** on checkout and login, so a scripted attack cannot exhaust stock or hammer the payment gateway.

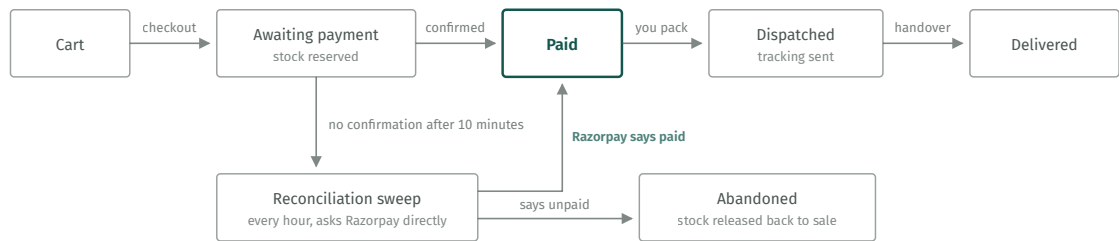
We prove this before launch, not after. In the final week we run a scripted load test simulating **2,000 concurrent shoppers and 100 checkouts a minute** for a sustained period, and we do not go live until it passes with the targets above intact. We also restore the database from backup into a scratch environment and check it comes back clean — an untested backup is not a backup. Both results are written up and handed to you.

MONTHLY INFRASTRUCTURE, ESTIMATED — BILLED TO YOUR OWN ACCOUNTS, NOT THROUGH JTACS		
ITEM	PER MONTH	NOTE
Application hosting	₹1,500–2,500	Scales with traffic; low at launch volumes
Managed PostgreSQL	₹800–1,600	Includes automated backups
Redis	₹400–800	Cache and background queue
Image storage and CDN	₹200–600	Grows with catalogue size
Monitoring and error tracking	₹0–800	Free tier is sufficient at launch
Domain name	~₹100	Billed yearly, about ₹1,200
Infrastructure total	₹3,000–6,400	No platform fee. No commission on sales.

On top of this you pay Razorpay only when a payment succeeds — **0% on UPI**, 2% plus GST on cards and net banking — and WhatsApp per message once Phase 3 is live. Estimates are based on published list prices at the time of writing and will be confirmed against real usage in month one.

Making sure the money is always right

The worst failure in an online shop is not a slow page. It is a customer who paid and whose order vanished. This is how that is prevented.



An order is never lost because a message went missing. Normally Razorpay confirms a payment to us within seconds. If that confirmation never arrives — a dropped connection, a moment of downtime — the order does not sit stranded: an hourly sweep asks Razorpay directly about every unconfirmed order and either completes it or releases the reserved stock. The customer's money and your stock are reconciled either way, without anyone noticing a problem.

THE SAFEGUARDS, PLAINLY

- ❑ **Card details never touch your system.** Payment happens on Razorpay's own page. We receive a yes or a no, never a card number — which removes an entire category of risk from your business.
- ❑ **Every message from Razorpay is signature-checked.** A forged "payment successful" message cannot create a paid order, because it will not carry Razorpay's signature.
- ❑ **A repeated confirmation cannot create a second order.** Payment services deliberately retry messages. Each one carries an identifier we record, so the same payment processed five times still produces exactly one order.
- ❑ **Stock cannot go negative.** Reserved at checkout, committed on payment, released automatically if payment never completes.
- ❑ **Every change is written to an audit trail** — who changed what, when. If a figure is ever questioned, it can be answered from the record rather than from memory.
- ❑ **Refunds go through Razorpay properly** and are recorded against the order, so your books and the gateway always agree.

Security, backups and keeping it running

Security

What protects the site

- Encrypted connection everywhere, enforced
- No card data stored, ever
- Passwords and API keys held in a managed secret store, never in the code
- Dashboard access by named account with two-factor authentication and roles — staff cannot see what owners see
- Rate limiting and bot protection on checkout and sign-in
- Dependencies scanned for known vulnerabilities on every change

Continuity

If something goes wrong

- Automated daily database backups with point-in-time recovery
- A restore rehearsed and verified before launch, not assumed
- Uptime monitoring that alerts within a minute of the site going down
- Error tracking, so faults are found before customers report them
- Every deployment reversible — a bad release is rolled back, not debugged live
- A written runbook so recovery does not depend on one person's memory

Being straight with you about ownership. On a rented platform, an outage is the platform's problem. On your own build it is yours, and the saving in Section 01 is the other side of exactly that coin. The measures above are how we keep that risk small: managed services wherever someone else can carry the burden, monitoring so problems surface early, and tested backups so the worst case is hours rather than a business. What none of it removes is the need for someone to be responsible for the system after launch — which is a commercial matter, and is dealt with in Section 12.

Compliance, built into the product

You hold the licences. Our job is to make sure the software cannot quietly put you in breach of them.

The mechanism, not a promise. Every declaration the law requires on a food listing is a **mandatory field on the product record**. A product with any of them missing **cannot be published to the storefront** — the system refuses. That way compliance does not depend on anyone remembering.

REQUIRED ON EVERY PRODUCT PAGE — ENFORCED BY THE SOFTWARE

DECLARATION	WHY IT IS REQUIRED
Net quantity	Legal Metrology (Packaged Commodities) Rules, as amended in February 2026, which specifically tightened what an online listing must display — country of origin in particular. Penalties are assessed per package , so a single non-compliant listing across a large consignment adds up quickly.
Retail sale price, inclusive of all taxes	
Name and address of the manufacturer or packer	
Date of manufacture or packing	
Best before / use by	
Country of origin	
Consumer care contact	
FSSAI licence number	Displayed on listings and in the site footer, as required of a food business selling online.
HSN code and GST rate per product	Roasted and ground coffee and most spices are 5% (HSN 0901 and 0904–0910); value-added blends can attract 12%. Set per product so invoices are correct at source.

ALSO HANDLED

- ☐ **GST invoices** raised automatically with a sequential, unbroken number series per financial year — generated in a way that cannot leave gaps when a payment fails.
- ☐ **Monthly GST and HSN export** from the Owner View, formatted for your accountant.
- ☐ **Store policy pages** — shipping, returns, refunds, privacy and terms. We draft them; please have them reviewed before launch.
- ☐ **Customer data kept to the minimum** needed to fulfil an order, with a documented process for deleting a customer's data on request.

Where our responsibility ends. JTACS builds and operates the software. We are not your legal, food-safety or tax advisor, and we do not verify the accuracy of the label data you supply — the system enforces that the fields are *filled*, not that they are *correct*. Product claims, licence validity and GST rates remain yours to confirm with your advisors.

How we will run the build

Six weeks in five phases, each ending in something you can see and sign off. Nothing moves forward until the previous gate is cleared.

What actually sets the date. The code is not the slow part. **Razorpay verification, Meta business verification, and your product photographs and descriptions are** — these run on other people's clocks, not ours. That is why Phase 0 starts everything in parallel on day one. Realistically, the timeline below holds if your catalogue material reaches us in week one; every week it is late is a week added at the end.

PHASE 0 · WEEK 1 · FOUNDATIONS

Accounts, infrastructure and content

Kick-off. Domain registered, hosting, database and code repository set up in your name. Razorpay account opened and verification filed. Meta business verification started, both WhatsApp numbers registered. We collect your product list, declarations, photographs and brand material.

Gate: accounts live, catalogue data received

PHASE 1 · WEEKS 2-3 · THE SHOP

Storefront, checkout and operations console

Commerce core deployed and configured. Storefront designed and built. Full catalogue loaded with every required declaration. Razorpay connected and tested with live one-rupee transactions across UPI, card and net banking, including a refund. Operations console secured and set up for your staff.

Gate: you place a real order on a preview link, end to end

PHASE 2 · WEEK 4 · YOUR VIEW

Owner dashboard and invoicing

Owner View built — daily and monthly sales, pending dispatch, top products, stock warnings, new versus repeat customers. GST invoice generation and the monthly accountant export. Content, policy pages and SEO completed.

Gate: dashboard figures reconcile against Razorpay for the test period

Community, in-chat ordering, order updates

Community created and structured on the free app. API number connected, message templates submitted and approved by Meta. In-chat order form built and wired to the dashboard. Automatic order-update messages switched on. Join links, QR codes and parcel inserts produced.

Gate: an order placed inside WhatsApp appears in the dashboard unaided

Load test, restore drill, training, launch

Load test at 2,000 concurrent shoppers and 100 checkouts a minute. Backup restored into a scratch environment and verified. Security review and dependency scan. Your corrections applied. Ninety-minute staff training, recorded. Runbook handed over. Go live.

Gate: load test and restore drill both pass, in writing

WHAT WE NEED FROM YOU

- ❑ **Product list with every declaration** — name, description, net quantity, MRP, manufacturer or packer details, packing date, best before, country of origin, HSN code. This is the item most likely to delay launch, so it is worth starting today.
- ❑ **Photographs** of each product, plus estate and process images for the story pages. Good photography sells produce; poor photography is the ceiling on your conversion rate.
- ❑ **Business documents** for Razorpay verification — PAN, GSTIN, FSSAI licence, business registration, cancelled cheque.
- ❑ **Two phone numbers** for WhatsApp, neither currently in use on a personal WhatsApp you wish to keep.
- ❑ **Your logo and brand material**, if you have them.
- ❑ **One decision-maker for thirty minutes a week.** Delay in approvals is the only thing that reliably moves the launch date.

What you own at the end

Yours outright

Accounts and data

Every account — domain, hosting, database, Razorpay, Meta, storage — is opened **in your name, on your card**, from day one. JTACS is added as a user and can be removed any day without the shop stopping. Your customer and order data is yours, exportable in a standard format whenever you ask.

Handed over

Code and documentation

The full source code in a repository under your account, with a **runbook** covering deployment, key rotation, backup restoration, reading the logs and what to do when the site is down. Plus a recorded ninety-minute training session for you and your staff.

Why the runbook matters more than it sounds. A custom system is only as safe as the knowledge of how to run it. Written down and handed over, that knowledge survives holidays, illness and changes of supplier — including us. We would rather you were never stuck than that you were dependent.

Decisions to confirm, and what is not included

TWO DECISIONS NEEDED BEFORE WE QUOTE

DECISION	THE OPTIONS	WHY IT MATTERS TO YOU
After-launch support	A monthly care agreement, or ad-hoc work charged as it arises.	A custom system needs someone to patch it, renew its certificates and fix faults. On a rented platform that is included in the rent; here it is a decision. Ad-hoc is cheaper until the week you need someone urgently.
Source code	Transferred to you outright, or licensed to you for this business.	Outright transfer costs more but leaves you free to take the system to any developer. A licence costs less upfront. Either way you keep the accounts, the data and a running shop.

NOT INCLUDED IN THIS PLAN

Product photography and videography · logo and brand identity design · copywriting beyond the product text you supply · paid advertising and its spend · delivery and courier arrangements, which you handle · Android or iOS apps · subscription and repeat-delivery

ordering · marketplace listings on Amazon or Flipkart · multi-language storefront · wholesale or dealer pricing tiers. Each can be quoted separately when you want it — several are natural additions once the shop is trading.

What happens next. Confirm the two decisions above and the expected order volume in Section 01, and we will issue the quotation and a build agreement against this plan. Nothing in this document is charged for, and nothing starts until you have both in hand and have approved them.

NOTES

This plan describes an intended build and the standards it will be held to; it is not a contract. Infrastructure costs are estimates based on published list prices in September 2026 and will vary with actual traffic and catalogue size. Third-party charges — hosting, Razorpay, Meta — are set by those companies, billed to you directly, and carry no JTACS margin. Capacity figures are design targets to be verified by load test before launch, as described in Section 06. Regulatory references are provided for context and are not legal advice. Sources consulted:

1. [Medusa — deployment documentation](#)
2. [Medusa — self-hosting considerations](#)
3. [Razorpay payment provider for Medusa](#)
4. [Razorpay — KYC documents by business type](#)
5. [Razorpay — WhatsApp payments merchant playbook, 2026](#)
6. [Meta — receiving payments via payment gateways on WhatsApp](#)
7. [WhatsApp Flows — in-chat forms, 2026](#)
8. [WhatsApp Flows — webhook payloads and response limits](#)
9. [FSSAI licensing for e-commerce food sellers](#)
10. [Legal Metrology \(Packaged Commodities\) Amendment Rules, 2026](#)
11. [GST rate on coffee powder, 2026](#)
12. [GST rate on spices and spice powders, 2026](#)